

Introduction to Legged Robotics: The Hexapod

M. J. Khojasteh

May 2025

1 Objective

This experiment explores a specific class of mobile robots commonly referred to as walking robots. A six-legged hexapod is used to demonstrate specific locomotion techniques that provide mobility within its terrain. Students will implement three common hexapod gaits for walking and create their own gait for turning. The joints of the hexapod are powered by servo motors which in turn are controlled by a SSC-32 servo controller board and an Arduino UNO controller board. A PING))) ultrasonic sensor is used to provide environmental feedback to the robot. The designed hexapod gaits and sensory feedback are integrated to provide a simple level of autonomy in the form of low-level obstacle detection and avoidance

Upon completion of this lab, students should be able to:

1. Understand how an Ultrasonic Sensor functions.
2. Execute different gaits, and understand the difference and utility of each.
3. Utilize the Ultrasonic sensor alongside gaits to implement object detection and avoidance.

2 Pre-Lab

There is no pre-lab for this lab.

3 Theory

Locomotion is the first challenge of a mobile robot, where its mechanics define how locomotion can be achieved. The six-legged mobile robot configuration is popular due to its statically stable walking abilities. For the particular hexapod used in this experiment, each leg has two degrees of freedom including knee flexion and hip abduction. This robot has less maneuverability in uneven terrain than its three (3+) degree of freedom counterparts, but performs well on flat terrain. Each of the six legs are powered by two independent

servo motors allowing the robot to lift a leg and make a stride simultaneously. Through sequential lifting and striding of the hexapod's legs it is possible to make the hexapod walk. The specific pattern of walking or locomotion chosen to provide the hexapod with mobility is referred to as its gait. There are many different types of gaits that exist for six-legged robots, but three of the most common are the **tripod**, **wave**, and **ripple**.

The control of all the links and joints of a six-legged mobile robot can be particularly taxing on a system. Therefore, the hexapod controls each of the servos independently with a SSC-32 servo controller via serial data communication. The serial communication is provided by a user written program downloaded to the Arduino microcontroller. The servo controller interprets the serial data into a series of **Pulse Width Modulated** signals (PWM) that in return determine the position of the servos. The SSC-32 servo controller is a very important feature of this robot. A servo controller is not necessary to control servos of this robot, as they can be controlled directly from the microcontroller. However, as the number of servos increase, the microprocessor can become consumed with servo control logic and leave no time for monitoring other sensors or I/O. Therefore, the advantage of the SSC-32 is that it performs all of the timing measures for the servos and leaves the Arduino microprocessor open to handle other tasks.

One of the most important requirements of an autonomous mobile robot is its ability to gain knowledge about its environment. A robot is able to perceive its environment by gathering measurements from sensors and interpreting the data. With a meaningful representation of its environment, a robot is able to move about its environment in an expected manner. The PING))) ultrasonic distance sensor is attached to the hexapod and is the robot's only way to perceive its environment. The PING))) ultrasonic distance sensor is an active sensor that provides precise, non-contact distance measurements. These sensors work by sending an ultrasonic burst into the environment, and returning a pulse that corresponds to the time of flight needed for the echo burst to return. The distance to the object can be calculated by the width of the echo pulse. With the ability to sense distances to objects in front of the PING))) sensor, the hexapod can be programmed to operate autonomously via simple obstacle detection and avoidance behaviors.

3.0.1 Servos

The servos of the hexapod, seen below in Figure 1, require a positive pulse of 1-2 ms in length with an off time of 5-20 ms. The on-time is used to position the output shaft of the servos while the off time is used for a rest. The left side of Figure 1 shows the relationship between the pulse length and the servo output shaft position. The relationship is proportional, so the output shaft can be positioned anywhere between the full left and full right positions. The original application of the servos required only a 90-degree angle of freedom. However, the extended 180-degree range is made available by using the SSC-32.

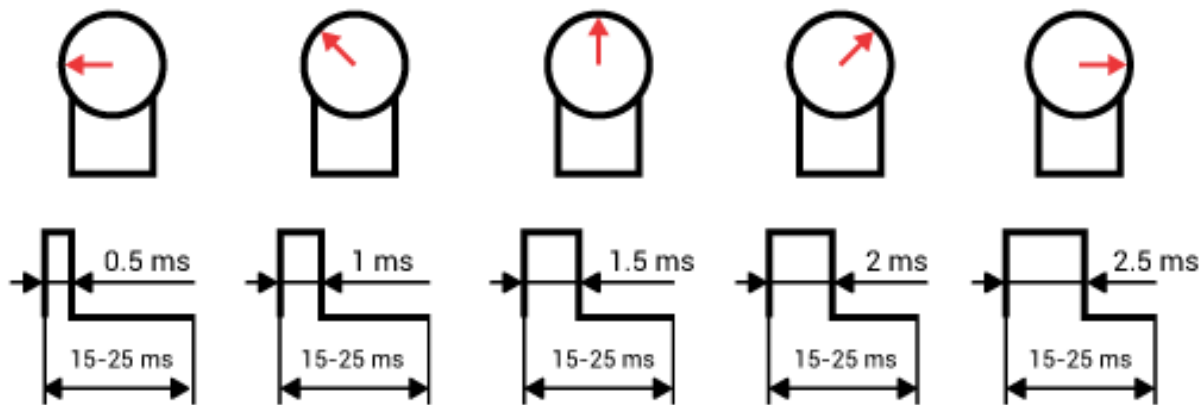


Figure 1: Servo Positions and Corresponding PWM Length

Make sure to not go beyond the mechanical stop of the servos, one servo may have a greater range than another, so just be careful.

3.0.2 Ultrasonic Sensors

A trigger pulse can be sent to the PING))) sensor, where the PING))) sensor emits a short 40 KHz burst. The burst can travel through the air at about 1130 feet/s and once it hits an object it bounces back to the sensor. While the burst is traveling towards an object the sensor concurrently provides an output pulse to the microcontroller. When the burst echo is received by the sensor the output pulse is terminated leaving an output pulse that is proportional to the distance to the object detected.

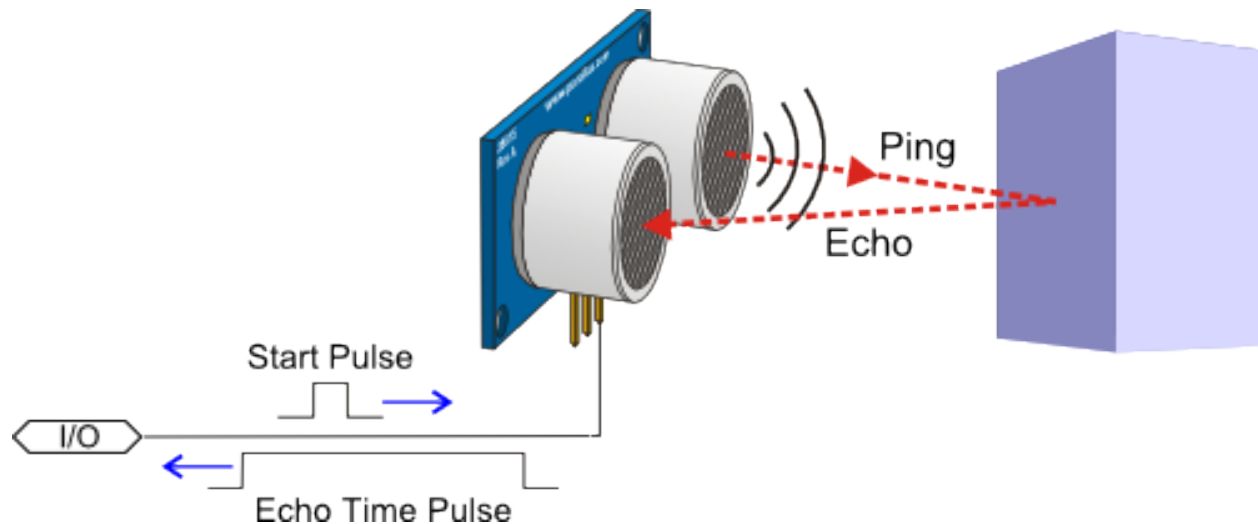


Figure 2: Overview of how an Ultrasonic Sensor Works

To calculate the distance of the detected object from the sensor the following equation can be used:

$$c = \frac{d}{t/2} \quad (1)$$

$$d = \frac{c}{2} * t \quad (2)$$

Where c is the speed of sound (343 m/s for this use case), d is the distance the wave traveled to hit the object, and t is the time it took for the wave to travel back. Note that the travel time is halved as t is measured as the time for the wave to travel to the object and back. In this case only the time to travel back is required to achieve the correct distance measurement.

3.0.3 Hexapod Gaits

The three most common hexapod gaits are called the **tripod**, **wave**, and **ripple**. Each gait follows a specific pattern of movement of the legs. There are advantages and disadvantages of each of the gaits specifically concerning stability and speed. In regards to stability, for a six-legged robot to be considered statically stable at least three of the six legs must remain on the ground at all times. It is possible to create dynamically stable gaits with less than three legs on the ground. These are typically faster than statically stable gaits but increase in control complexity by requiring the robot to be in motion.

1. **Tripod Gait** - The tripod gait or alternating tripod gait is characterized as one middle leg on one side and two non-adjacent legs on the other side of the body lifted and moved forward at the same time; while the other three legs remain on the ground to keep the robot statically stable. This is the fastest of the three gaits and converges in two steps, but also the least stable. For video demonstration watch this video [HERE](#).
2. **Wave Gait** - The wave gait can be characterized by lifting and moving only one leg at a time in a "wave-like" fashion. This gait is the most stable of all the gaits since, at any given instance five legs are in contact with the ground. However, this is a very slow walking pattern, since it converges in six steps, and is most typically used in very rough terrain. For video demonstration watch this video [HERE](#).
3. **Ripple Gait** - The ripple gait is very similar to the wave gait. Each side of the hexapod performs its own wave gait simultaneously, with one side being 180° out of phase with the other side. The ripple gait is not as fast as the tripod gait since it converges in three steps, but it provides more stability with four legs on the ground at any given time. The ripple gait is most commonly used with legged-robots that have more than six legs. For video demonstration watch this video [HERE](#)

In the below figures and tables you will see the positions of the individual legs for each step of all three of the gaits. Note that if a step converges in say three steps, each leg will have three possible positions that define the entire range of its motion. Additionally each step is best executed if decomposed into three sub-steps that sequentially lift the required legs (lift), move all legs (stride), and then place back down the lifted legs (drop). For each of the legs shown in Figure 3 there are two numbers listed either labeled with *L* or *S*. The number listed beside the *L* corresponds to the servo number that will enable that leg to perform a lift. The number listed beside the *S* corresponds to the number that will enable that leg to perform a stride. The tables provide the gait pattern where a 1 represents the legs that are NOT touching the ground and a 0 represents the legs that are touching the ground.

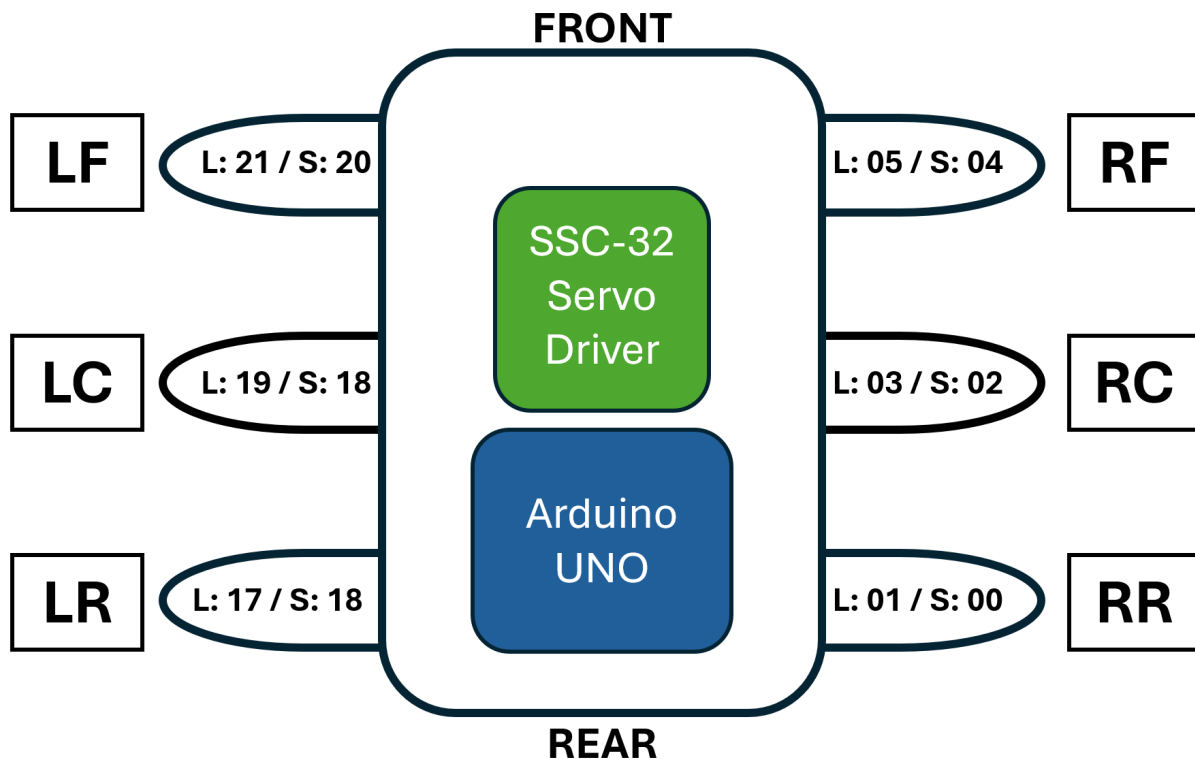


Figure 3: Schematic of Naming Convention and Servo Pin-Outs for the Hexapod

STEP #	TRIPOD GAIT - LEG POSITIONS					
	LF	LC	LR	RF	RC	RR
1	1	0	1	0	1	0
2	0	1	0	1	0	1

Table 1: Tripod Gait Sequence Showing Leg Positions for each Step of the Hexapod

STEP #	WAVE GAIT - LEG POSITIONS					
	LF	LC	LR	RF	RC	RR
1	0	0	1	0	0	0
2	0	1	0	0	0	0
3	1	0	0	0	0	0
4	0	0	0	0	0	1
5	0	0	0	0	1	0
6	0	0	0	1	0	0

Table 2: Wave Gait Sequence Showing Leg Positions for each Step of the Hexapod

STEP #	RIPPLE GAIT - LEG POSITIONS					
	LF	LC	LR	RF	RC	RR
1	1	0	0	0	0	1
2	0	1	0	1	0	0
3	0	0	1	0	1	0

Table 3: Ripple Gait Sequence Showing Leg Positions for each Step of the Hexapod

The hexapod motion or heading depends on the stride angle, for instance unequal stride angles will cause the hexapod to drift or move sideward instead of following a straight path of motion.

4 Introduction

4.1 Hexapod Platform

The hexapods that we will be using in this lab rely on the Arduino UNO board as its main compute. Arduino is an open-source electronics platform that is based on easy to use hardware and software. Arduino boards are meant to be able to interface with a multitude of sensors which is quite useful for a lab such as this. To code the Arduino UNO we will rely on their built in Arduino Software IDE, in which you will code in what many call "Arduino C". In many ways it is extremely similar to classic C and C++, however it has its own quirks that differentiate it.

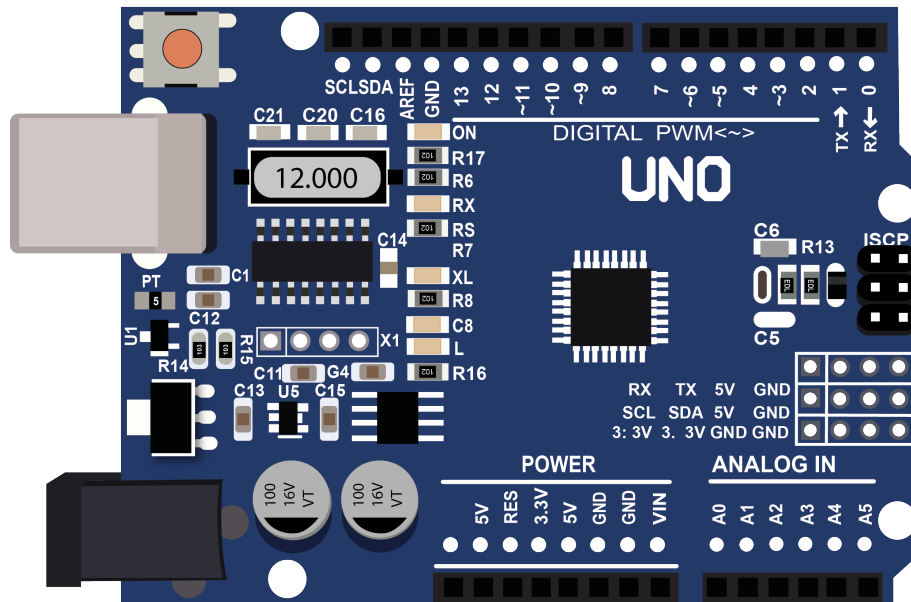


Figure 4: Arduino UNO Board

4.1.1 Servo Controller

There is a servo controller board SSC-32 that is connected to the Arduino. The SSC-32 is used specifically to control the servos, 12 for the legs and 1 for the PING))) Ultrasonic sensor.

4.1.2 Servos

The servos that are used in the Hexapod are the Hitec HS-422 paired in two for each leg to provide the striding and lift for each leg. Additionally as stated in the previous subsection there is another servo used for the ultrasonic sensor.

4.1.3 Power

The hexapod is equipped with a 7.2V rechargeable battery pack. Make sure that at the end of the lab session you plug the batteries back into the charging station.

4.2 Programming the Robot

1. Download the required documents and files which reside within this Lab folder from MyCourses.
2. Add the SSC-32 library provided
3. Run the example code provided named `home.ino`
 - The example code provided uses the SCC-32 library files to control the servo motors.

IMPORTANT FUNCTIONS AND VARIABLE DEFINITIONS:

- The following defines the pin number to which the servos are connected on the SSC-32 board, which should match with the schematic shown in Figure 3.

Example Code

```
int channel[] = {servo_pin_1, ... servo_pin_n};
```

- The following above defines the T_{on} of the servo connected to each pin.

Example Code

```
int homeframe[] = {servo_pin_1_Ton, ... servo_pin_n_Ton};
```

- The following enables the servos connected to the defined pins in the `channel[]` array.

Example Code

```
SSC32_obj.enableServos(channel);
```

- The following sets the servo positions based on the timing values defined in the `homeframe[]` array.

Example Code

```
SSC32_obj.setFrame(channel, homeframe);
```

NOTE: The range of time values is 500-2500. Any value beyond the limit may damage the servos permanently and is not advisable. The hexapod home position is ideally obtained by setting the time value to 1500, however as you'll see the home position may vary servo to servo.

4.3 Additional Notes

1. Make sure to check that every leg is straight and hits the ground in the home position.
2. If you are in need of more reference material for issues concerning program syntax and keywords you can find the Arduino reference page [HERE](#)

5 Deliverables

1. Understand the structure of the given example code or create your own and wire the servos to the SSC-32 board using the provided code as an initial guide.
2. Program the hexapod to go to its home position.
3. Program the hexapod to walk forward in a straight line using:
 - (a) Tripod Gait
 - (b) Ripple Gait
 - (c) Wave Gait
4. Create your own turning gait
 - (a) Demonstrate turning left and right.
5. Demonstrate some simple object avoidance
 - (a) *Hint: Walk forward until you detect an object $\leq$ threshold then act accordingly.*
6. **EXTRA CREDIT:** Create your own custom gait. (e.g. implement walking backwards)

6 Questions

1. Why must the servos on each leg be independent of each other?
2. What are the advantages and the disadvantages of a legged robot?
3. Compare and contrast the three gaits specified.
4. What common mobile robot problem can be solved by utilizing an ultrasonic sensor?

7 Report

Make sure to submit a tech memo which follows the sekeleton/guidelines that are located in MyCourses.

Your report should include the following:

1. How your custom turning gait works.
2. How sonar measurement conversion is done.
3. Flow chart of obstacle detection and avoidance.
4. Answers to all of the questions.
5. A summary of what was accomplished and learned within the lab.